

ZeRO-1: A Concrete Walkthrough with Actual Numbers

1. Our Tiny Transformer Setup

We define a deliberately tiny model so every matrix fits on paper.

Hidden dimension (d)	= 4
Number of attention heads	= 2 → head dimension $d_k = 4/2 = 2$
FFN inner dimension	= 16 (4× expansion)
Vocab size	= 8
Sequence length (T)	= 3 (3 tokens)
Number of GPUs	= 2 (GPU-0 and GPU-1)

2. Listing Every Parameter (with shapes)

Following the architecture diagram (LN1 → Attention → residual → LN2 → FFN → residual → W_vocab → softmax):

Layer	Name	Shape	#Elements
LayerNorm 1	γ_1	(4,)	4
	β_1	(4,)	4
Attention:			
Query weights	W_q	(4, 4)	16
Key weights	W_k	(4, 4)	16
Value weights	W_v	(4, 4)	16
Output proj	W_o	(4, 4)	16
LayerNorm 2	γ_2	(4,)	4
	β_2	(4,)	4
FFN:			
Up-projection	W_1	(4, 16)	64
Bias 1	b_1	(16,)	16
Down-projection	W_2	(16, 4)	64
Bias 2	b_2	(4,)	4
Output head	W_{vocab}	(4, 8)	32
TOTAL PARAMETERS:			260

3. Concrete Parameter Values (Initialised)

Let's assign actual numbers. We'll show a few key matrices explicitly.

W_q (Query weights) — shape (4, 4):

```
W_q = [ 0.12  0.34 -0.21  0.05 ]  
      [-0.15  0.22  0.11 -0.08 ]  
      [ 0.30 -0.10  0.18  0.27 ]  
      [-0.05  0.14 -0.33  0.09 ]
```

W_k (Key weights) — shape (4, 4):

```
W_k = [ 0.20 -0.11  0.07  0.15 ]  
      [ 0.03  0.28 -0.14  0.10 ]  
      [-0.22  0.06  0.31 -0.05 ]  
      [ 0.17 -0.09  0.02  0.24 ]
```

W_v (Value weights) — shape (4, 4):

```
W_v = [ 0.08  0.19 -0.12  0.25 ]  
      [-0.07  0.33  0.04 -0.16 ]  
      [ 0.21 -0.03  0.15  0.11 ]  
      [-0.14  0.08  0.26 -0.02 ]
```

W₁ (FFN up-projection) — shape (4, 16): (showing first 4 of 16 columns)

```
W_1 = [ 0.10  0.05 -0.22  0.13  ... ]  (4 rows × 16 cols = 64 values)  
      [-0.08  0.17  0.03 -0.11  ... ]  
      [ 0.25 -0.06  0.14  0.09  ... ]  
      [-0.03  0.21 -0.07  0.18  ... ]
```

W_{vocab} (output head) — shape (4, 8):

```
W_vocab = [ 0.11 -0.05  0.18  0.03 -0.14  0.22 -0.09  0.07 ]  
          [-0.06  0.13  0.02 -0.17  0.08  0.04  0.20 -0.11 ]  
          [ 0.15 -0.02  0.09  0.21 -0.08  0.12 -0.03  0.16 ]  
          [-0.10  0.07 -0.13  0.05  0.19 -0.06  0.14  0.01 ]
```

LayerNorm parameters:

```
γ1 = [1.0, 1.0, 1.0, 1.0]  (initialized to ones)  
β1 = [0.0, 0.0, 0.0, 0.0]  (initialized to zeros)
```

$$\gamma_2 = [1.0, 1.0, 1.0, 1.0]$$

$$\beta_2 = [0.0, 0.0, 0.0, 0.0]$$

4. Memory Accounting: Why ZeRO-1 Matters

For each of the 260 parameters, we must store:

What	Precision	Bytes/element
------	-----------	---------------

Parameter (for fwd/bwd)	BF16	2
-------------------------	------	---

Gradient (after backward)	BF16	2
---------------------------	------	---

Optimizer states (Adam)

Master copy of parameter	FP32	4
--------------------------	------	---

First moment m	FP32	4
----------------	------	---

Second moment v	FP32	4
-----------------	------	---

TOTAL per element:	16 bytes
--------------------	----------

Without ZeRO (standard data-parallel):

Per GPU = 260 elements $\times$ 16 bytes = 4,160 bytes

GPU-0: 4,160 bytes (full params + full grads + full optimizer)

GPU-1: 4,160 bytes (full params + full grads + full optimizer)

↑↑↑

100% redundancy here!

TOTAL across cluster: 8,320 bytes

With ZeRO-1 (partition optimizer states only):

Per GPU:

Parameters (full, BF16): $260 \times 2 = 520$ bytes

Gradients (full, BF16): $260 \times 2 = 520$ bytes

Optimizer (1/2 of total): $130 \times 12 = 1,560$ bytes ← THE SAVING

Per GPU total: 2,600 bytes (was 4,160)

SAVING: 37.5% less memory per GPU

For a real model (e.g., 7B params), optimizer states dominate:

- Without ZeRO: $7B \times 16 = 112$ GB per GPU
- ZeRO-1 on 8 GPUs: $7B \times 4$ (params+grads) + $7B \times 12/8$ (optimizer) = $28 + 10.5 = \mathbf{38.5}$ GB per GPU (vs 112 GB)

5. The Partition: Who Owns What

We flatten all 260 parameters into a single 1D vector and split it in half:

Flat parameter vector (260 elements):

[$\gamma_1(4)$ | $\beta_1(4)$ | $W_q(16)$ | $W_k(16)$ | $W_v(16)$ | $W_o(16)$ | $\gamma_2(4)$ | $\beta_2(4)$ | $W_1(64)$ | $b_1(16)$ | $W_2(64)$ | $b_2(4)$ | $W_{\text{vocab}}(32)$]

indices 0..129 → GPU-0 owns optimizer indices 130..259 → GPU-1 owns optimizer

GPU-0 is responsible for optimizer states of:

$\gamma_1, \beta_1, W_q, W_k, W_v, W_o, \gamma_2, \beta_2$, and the first 2 rows of W_1
= 130 elements

GPU-0 stores:

$m_0[130]$ (first moments, FP32) ← 520 bytes
 $v_0[130]$ (second moments, FP32) ← 520 bytes
 $p_0[130]$ (master params, FP32) ← 520 bytes

1,560 bytes of optimizer state

GPU-1 is responsible for optimizer states of:

Remaining rows of $W_1, b_1, W_2, b_2, W_{\text{vocab}}$
= 130 elements

GPU-1 stores the corresponding $m_1[130], v_1[130], p_1[130]$.

But BOTH GPUs store:

Full parameters $\theta[260]$ in BF16 (needed for forward & backward)
Full gradients $g[260]$ in BF16 (computed during backward, before reduce-scatter)

6. Walking Through One Training Step

Input

Our 3-token input sequence, after embedding lookup:

$x_0 = [0.5, -0.3, 0.8, 0.1]$ $\leftarrow$ token 0 embedding (d=4)
[0.2, 0.7, -0.1, 0.4] $\leftarrow$ token 1 embedding
[-0.6, 0.3, 0.5, -0.2] $\leftarrow$ token 2 embedding

Shape: (3, 4) = (T, d)

GPU-0 gets micro-batch A (e.g., "The cat sat"), GPU-1 gets micro-batch B (e.g., "A dog ran"). Both have different x_0 values but use the same model parameters.

STEP 1: Forward & Backward Pass (Local Compute)

Each GPU independently runs the full model. Let's trace GPU-0:

LayerNorm 1:

$$x_1 = \text{LN}(x_0) = \gamma_1 \odot (x_0 - \mu) / \sigma + \beta_1$$

For token 0: $\mu = (0.5 - 0.3 + 0.8 + 0.1)/4 = 0.275$

$$\sigma = \sqrt{\text{var} + \epsilon} \approx 0.398$$

$$\begin{aligned} x_1[0] &= 1.0 \times (x_0[0] - 0.275)/0.398 + 0.0 \\ &= [0.565, -1.445, 1.319, -0.439] \end{aligned}$$

Attention (2 heads, d_k = 2):

Compute Q, K, V for all 3 tokens:

$$Q = x_1 \cdot W_q \rightarrow (3, 4) \times (4, 4) = (3, 4)$$

$$K = x_1 \cdot W_k \rightarrow (3, 4) \times (4, 4) = (3, 4)$$

$$V = x_1 \cdot W_v \rightarrow (3, 4) \times (4, 4) = (3, 4)$$

For token 0, computing Q:

$$\begin{aligned}
Q[0] &= x_1[0] \cdot W_q \\
&= [0.565, -1.445, 1.319, -0.439] \times \begin{bmatrix} 0.12 & 0.34 & -0.21 & 0.05 \\ -0.15 & 0.22 & 0.11 & -0.08 \\ 0.30 & -0.10 & 0.18 & 0.27 \\ -0.05 & 0.14 & -0.33 & 0.09 \end{bmatrix} \\
&= [(0.565 \times 0.12) + (-1.445 \times -0.15) + (1.319 \times 0.30) + (-0.439 \times -0.05), \\
&\quad (0.565 \times 0.34) + (-1.445 \times 0.22) + (1.319 \times -0.10) + (-0.439 \times 0.14), \\
&\quad \dots] \\
&= [0.068 + 0.217 + 0.396 + 0.022, \\
&\quad 0.192 - 0.318 - 0.132 - 0.061, \\
&\quad \dots] \\
&= [0.703, -0.319, \dots]
\end{aligned}$$

Split into 2 heads (first 2 dims = head 0, last 2 dims = head 1):

$$\begin{aligned}
\text{Head 0: } Q_0 &= Q[:, 0:2], \quad K_0 = K[:, 0:2], \quad V_0 = V[:, 0:2] \\
\text{Head 1: } Q_1 &= Q[:, 2:4], \quad K_1 = K[:, 2:4], \quad V_1 = V[:, 2:4]
\end{aligned}$$

Attention scores for head 0:

$$\begin{aligned}
A_0 &= \text{softmax}(Q_0 \cdot K_0^T / \sqrt{2}) \rightarrow (3, 3) \text{ attention matrix} \\
\text{E.g., } A_0[0,0] &= Q_0[0] \cdot K_0[0] / \sqrt{2} = (\text{dot product}) / 1.414
\end{aligned}$$

Then: $x_{\text{attn}} = \text{Concat}(A_0 \cdot V_0, A_1 \cdot V_1) \cdot W_o \rightarrow \text{shape } (3, 4)$

Residual + LN2 + FFN + Residual:

$$\begin{aligned}
x_{\text{res1}} &= x_0 + x_{\text{attn}} && \rightarrow (3, 4) \\
x_2 &= \text{LN2}(x_{\text{res1}}) && \rightarrow (3, 4) \\
x_{\text{ffn}} &= \text{ReLU}(x_2 \cdot W_1 + b_1) \cdot W_2 + b_2 && \rightarrow (3, 4) \\
&\quad \uparrow (3,4) \times (4,16) = (3,16) \quad \uparrow (3,16) \times (16,4) = (3,4) \\
x_{\text{final}} &= x_{\text{res1}} + x_{\text{ffn}} && \rightarrow (3, 4)
\end{aligned}$$

Output + Loss:

$$\begin{aligned}
\text{logits} &= x_{\text{final}} \cdot W_{\text{vocab}} \rightarrow (3, 4) \times (4, 8) = (3, 8) \\
z &= \text{softmax}(\text{logits}) \rightarrow (3, 8) \text{ probabilities over vocab} \\
\text{Loss} &= \text{CrossEntropy}(z, \text{targets}) \rightarrow \text{scalar}
\end{aligned}$$

Backward Pass:

Backprop computes $\partial \text{Loss} / \partial \theta$ for every parameter. After backward, GPU-0 holds:

```
g_q^(A) =  $\partial \text{Loss}_A / \partial W_q$  → shape (4,4), 16 gradient values (in BF16)
g_k^(A) =  $\partial \text{Loss}_A / \partial W_k$  → shape (4,4)
g_v^(A) =  $\partial \text{Loss}_A / \partial W_v$  → shape (4,4)
g_o^(A) =  $\partial \text{Loss}_A / \partial W_o$  → shape (4,4)
g_W1^(A) =  $\partial \text{Loss}_A / \partial W_1$  → shape (4,16)
g_W2^(A) =  $\partial \text{Loss}_A / \partial W_2$  → shape (16,4)
...and all other gradients
```

Say concretely:

GPU-0 computed (from micro-batch A):

```
g_q^(A) = [ 0.023 -0.011  0.045 -0.008 ]
          [-0.031  0.019 -0.007  0.014 ]
          [ 0.012 -0.028  0.033 -0.005 ]
          [-0.016  0.009 -0.021  0.038 ]
```

GPU-1 computed (from micro-batch B):

```
g_q^(B) = [ 0.017 -0.025  0.031 -0.013 ]
          [-0.009  0.041 -0.018  0.006 ]
          [ 0.028 -0.014  0.022 -0.035 ]
          [-0.020  0.016 -0.012  0.027 ]
```

Both GPUs now hold 260 gradient values (one per parameter) in BF16.

STEP 2: Reduce-Scatter (Communicate Gradients)

This is the crucial communication step. We need to **average** the gradients from both GPUs, but instead of giving everyone the full average (that would be all-reduce), we use **reduce-scatter**: each GPU receives only the averaged gradient for its assigned slice.

Flatten all 260 gradients into a vector and split:

Full gradient vector (260 elements):

```
[g_γ1 | g_β1 | g_W_q | g_W_k | ... ]
```

```
←──────── slice 0 (130) ─────────→ ←──────── slice 1 (130) ─────────→
```

Reduce-scatter result:

GPU-0 receives: $\text{avg_g}[0:130] = (g^A[0:130] + g^B[0:130]) / 2$

GPU-1 receives: $\text{avg_g}[130:260] = (g^A[130:260] + g^B[130:260]) / 2$

Concretely for W_q (which falls in GPU-0's slice):

GPU-0 ends up with:

$$\text{avg_g_q} = (\mathbf{g_q}^{\text{(A)}} + \mathbf{g_q}^{\text{(B)}}) / 2$$

$$= ([0.023 \ -0.011 \ 0.045 \ -0.008] \ [0.017 \ -0.025 \ 0.031 \ -0.013]) / 2 \\ ([-0.031 \ 0.019 \ -0.007 \ 0.014] + [-0.009 \ 0.041 \ -0.018 \ 0.006]) \\ ([0.012 \ -0.028 \ 0.033 \ -0.005] \ [0.028 \ -0.014 \ 0.022 \ -0.035]) \\ ([-0.016 \ 0.009 \ -0.021 \ 0.038] \ [-0.020 \ 0.016 \ -0.012 \ 0.027])$$

$$= [0.020 \ -0.018 \ 0.038 \ -0.0105] \\ [-0.020 \ 0.030 \ -0.0125 \ 0.010] \\ [0.020 \ -0.021 \ 0.0275 \ -0.020] \\ [-0.018 \ 0.0125 \ -0.0165 \ 0.0325]$$

GPU-1 does NOT have avg_g_q! It only has the averaged gradients for indices 130–259.

Similarly, GPU-1 has the averaged gradient for $\mathbf{W}_{\text{vocab}}$ (which falls in its slice), and GPU-0 does not.

STEP 3: Optimizer Step (Update Local Slice)

Now each GPU runs Adam **only on its 130-element slice**, using its local optimizer states.

GPU-0 updates $\mathbf{W}_q$ (Adam step, iteration $t=1$):

Hyperparameters: $\text{lr}=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1\text{e-}8$

For each element of $\mathbf{W}_q$, e.g., position $[0,0]$:

Current master param (FP32): $p = 0.12$

Averaged gradient: $g = 0.020$

Update first moment:

$$\begin{aligned} m_{\text{new}} &= \beta_1 \times m_{\text{old}} + (1 - \beta_1) \times g \\ &= 0.9 \times 0.0 + 0.1 \times 0.020 \\ &= 0.002 \end{aligned}$$

Update second moment:

$$\begin{aligned} v_{\text{new}} &= \beta_2 \times v_{\text{old}} + (1 - \beta_2) \times g^2 \\ &= 0.999 \times 0.0 + 0.001 \times 0.0004 \\ &= 0.0000004 \end{aligned}$$

Bias-corrected estimates ($t=1$):

$$\begin{aligned} \hat{m} &= m_{\text{new}} / (1 - \beta_1^t) = 0.002 / 0.1 = 0.02 \\ \hat{v} &= v_{\text{new}} / (1 - \beta_2^t) = 0.0000004 / 0.001 = 0.0004 \end{aligned}$$

Parameter update:

$$\begin{aligned} p_{\text{new}} &= p - lr \times \hat{m} / (\sqrt{\hat{v}} + \epsilon) \\ &= 0.12 - 0.001 \times 0.02 / (\sqrt{0.0004} + 1e-8) \\ &= 0.12 - 0.001 \times 0.02 / 0.02 \\ &= 0.12 - 0.001 \\ &= 0.119 \end{aligned}$$

Cast back to BF16: $W_q[0,0] = \text{BF16}(0.119) \approx 0.1191$

GPU-0 repeats this for all 130 elements in its slice. After this step:

GPU-0 state:

- ✓ Updated params slice [0:130] in BF16 (new values)
- ✓ Updated $m_0[130]$, $v_0[130]$, $p_0[130]$ in FP32
- ✗ Params slice [130:260] still has OLD values

GPU-1 state:

- ✓ Updated params slice [130:260] in BF16 (new values)
- ✓ Updated $m_1[130]$, $v_1[130]$, $p_1[130]$ in FP32
- ✗ Params slice [0:130] still has OLD values

The model is now inconsistent — each GPU has half new, half old parameters!

STEP 4: All-Gather (Sync New Parameters)

Each GPU broadcasts its freshly updated parameter slice to all others:

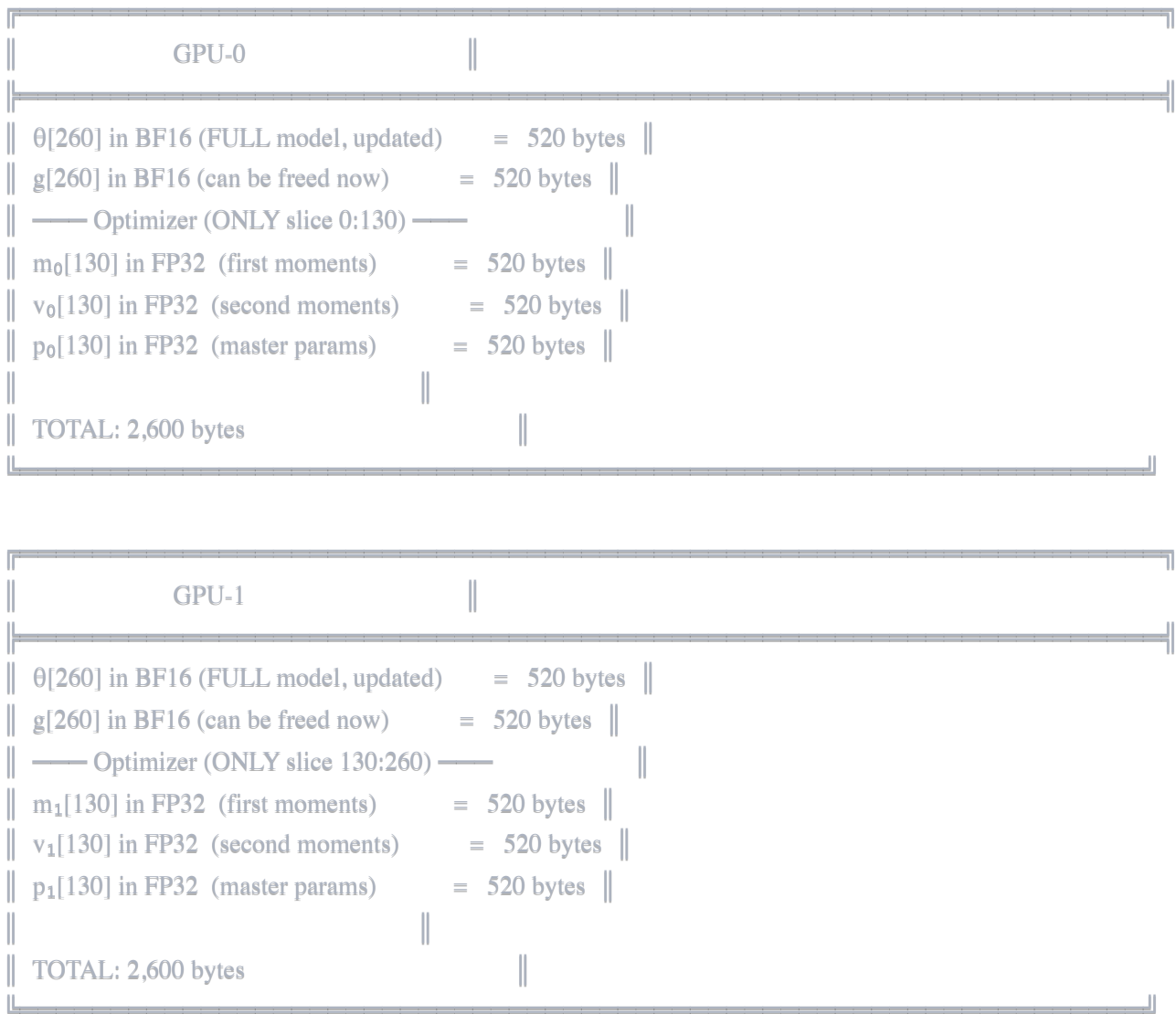
GPU-0 sends: new_params[0:130] → GPU-1
GPU-1 sends: new_params[130:260] → GPU-0

After all-gather:
GPU-0 has: [new_params[0:130] | new_params[130:260]] ← FULL updated model
GPU-1 has: [new_params[0:130] | new_params[130:260]] ← FULL updated model

Concretely, W_q is now identical on both GPUs:

W_q (updated, both GPUs) =
[0.119 0.341 -0.211 0.051] (values shifted slightly
[-0.149 0.219 0.111 -0.081] by the Adam update)
[0.299 -0.099 0.179 0.271]
[-0.049 0.139 -0.329 0.089]

STEP 5: End of Step — What Lives Where



Compare to standard data-parallel (no ZeRO): 4,160 bytes per GPU.

7. Communication Cost Analysis

Standard All-Reduce (no ZeRO):
Each GPU sends and receives 260 gradient values.
Communication volume: $2 \times 260 \times 2$ bytes = 1,040 bytes
(ring all-reduce: send + receive)

ZeRO-1:
Reduce-Scatter: each GPU sends 130 values, receives 130 values
 = $260 \times 2 = 520$ bytes
All-Gather: each GPU sends 130 values, receives 130 values
 = $260 \times 2 = 520$ bytes
TOTAL: 1,040 bytes

The communication volume is identical. ZeRO-1 saves memory for free (no extra communication cost), because a standard ring all-reduce is already internally implemented as a reduce-scatter followed by an all-gather. ZeRO-1 simply inserts the local optimizer step between these two phases.

8. Scaling to a Real Model: 7B Parameters on 8 GPUs

	No ZeRO	ZeRO-1 (8 GPUs)
Params (BF16):	14.0 GB	14.0 GB
Gradients (BF16):	14.0 GB	14.0 GB
Optimizer m (FP32):	28.0 GB	3.5 GB (+8)
Optimizer v (FP32):	28.0 GB	3.5 GB (+8)
Master params (FP32):	28.0 GB	3.5 GB (+8)
TOTAL per GPU:	112.0 GB	38.5 GB
Saving:	—	65.6%

This is why ZeRO-1 is the default in frameworks like DeepSpeed — it eliminates the majority of per-GPU memory usage (the optimizer states) with zero additional communication overhead.

9. Summary: The Key Insight

Standard Data-Parallel:

Every GPU stores: full params + full grads + full optimizer

→ 16 bytes per parameter per GPU

ZeRO Stage 1:

Every GPU stores: full params + full grads + $(1/N)$ th optimizer

→ $(4 + 12/N)$ bytes per parameter per GPU

The trick:

1. After backward, reduce-scatter gives each GPU only its gradient slice
2. Each GPU runs Adam on only its slice (cheap, local)
3. All-gather broadcasts the updated parameter slices
4. Communication is the SAME as standard all-reduce (just reordered)
5. Memory saving = $(N-1)/N$ of the optimizer states — essentially free